

PERSONAL LEARNING NOTES

AI Agents — Structured Learning Notes

Hugging Face Agents Course — Full Course Reference (Units 0-4 + Bonus 1-3)

Rewritten in my own structure for revision · Source: huggingface.co/learn/agents-course

Format: Overview · Why It Matters · Core Concepts · Example · Developer Notes · Quick Revision · Common Mistakes

📖 Table of Contents

00	Onboarding — Course Structure, Certification & Tools
01	Agent Fundamentals — Thought → Action → Observation
02	Frameworks Overview — smolagents vs LangGraph vs LlamaIndex
02.1	smolagents — CodeAgent framework
02.2	LlamaIndex — Agents over your data
02.3	LangGraph — Controllable agent graphs
03	Use Case: Agentic RAG
04	Final Project — GAIA Benchmark Agent
B1	Bonus — Fine-tuning an LLM for Function-calling
B2	Bonus — Agent Observability & Evaluation
B3	Bonus — Agents in Games (Pokémon)
★	Master Cheat Sheet — All Units in One Page

Onboarding — Welcome to the Course

Course structure, certification paths, tools, and how the course is run.

OVERVIEW

The Agents Course takes you from **beginner to expert** in understanding, using, and building AI Agents. It mixes theory (Unit 1), hands-on framework practice (Unit 2: smolagents, LlamaIndex, LangGraph), a real use-case (Unit 3: Agentic RAG), and a final certifiable project (Unit 4: GAIA benchmark agent). There are also 3 bonus units on fine-tuning, observability, and game-playing agents.

WHY IT MATTERS

Agents are the natural next step after "just prompting" an LLM — they let a model **act** in the world (call tools, browse, run code, query data) instead of only producing text. Understanding the fundamentals here transfers to any agent framework you'll meet later (LangChain, CrewAI, AutoGen, OpenAI Agents SDK, etc).

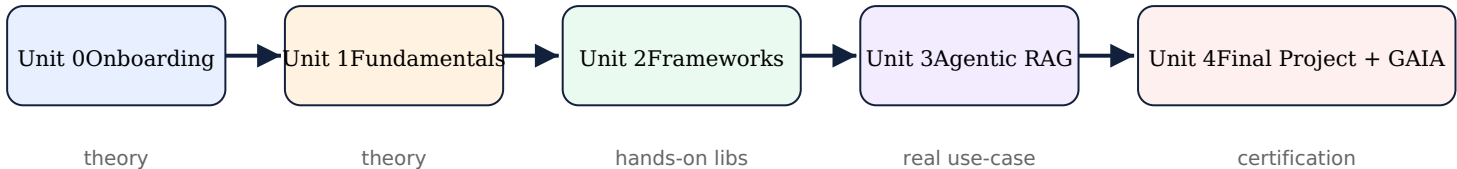
CORE CONCEPTS

Chapter	Topic	What it covers
0	Onboarding	Tools, platforms, syllabus, sign-up
1	Agent Fundamentals	Tools, Thoughts, Actions, Observations, LLMs, chat templates
2	Frameworks	smolagents, LangGraph, LlamaIndex
3	Use Cases	Real-life agent build — Agentic RAG
4	Final Assignment	Build + benchmark an agent on GAIA, get certified

Two certificates:

- **Certificate of Fundamentals** — complete Unit 1 only.
- **Certificate of Completion** — Unit 1 + one use-case assignment + final GAIA challenge (score ≥ 30%).

Pace: ~1 chapter/week, 3-4 hrs/week. **Prerequisites:** basic Python + basic LLM knowledge. **Tools needed:** a computer with internet + a free Hugging Face account.



DEEP DIVE

Content format per unit: Foundational (theory) → Hands-on (pre-configured Hugging Face Space) → Use-case assignment (apply it to a problem you pick) → Challenge (compete/compare on a leaderboard). **Certification detail:** the Fundamentals certificate only needs Unit 1; the Completion certificate additionally needs one use-case assignment (from Units 2/3) plus a passing GAIA score in Unit 4 — and there's no deadline, so it can be done at your own pace across multiple sittings.

KEY TAKEAWAY

This is a *practice-first* course: theory in Unit 1 is minimal on purpose, because agents are best understood by building them. Audit mode is fine — certification is optional and free.

DEVELOPER NOTES

- Create a free Hugging Face account before starting — you'll push models/Spaces throughout.
- Join the Discord study group for accountability ([#agents-course-questions](#)).
- Bonus units (fine-tuning, observability, games) are optional but great for portfolio projects.

QUICK REVISION

Onboarding → 5 chapters (0-4) + 3 bonus units → 2 certificate types → self-paced, free, hands-on.

Agent Fundamentals

What an Agent is, how LLMs think in text, and the Thought → Action → Observation cycle.

OVERVIEW

An **Agent** is a system that uses an **LLM as its "brain"** to reason about a goal, decide on **Actions** (usually via **Tools**), observe the results, and repeat until the goal is achieved. This unit covers: what agents are, LLM basics (next-token prediction, special tokens, chat templates), how Tools are described to an LLM, and the core **Thought → Action → Observation** loop (ReAct-style).

WHY IT MATTERS

A plain LLM only predicts text — it can't check today's weather, run code, or book a flight. Wrapping an LLM in an agent loop that can call **tools** and use the results turns a "text predictor" into a system that can **act on the real world**. Every framework (smolagents, LangGraph, LlamaIndex, LangChain, AutoGPT...) is just a different implementation of this same loop.

CORE CONCEPTS

1. What is an Agent?

Agent = LLM (brain) + Tools (hands) + a loop that lets it reason, act, and observe repeatedly, instead of answering in one shot.

Type	Description	Example
Simple Processor (no agency)	Output has no influence on the program flow	Basic chatbot, translator
Router	LLM output decides basic control flow (if/else style branch)	Intent classifier
Tool Caller	LLM decides which function to call + arguments	Weather-fetching bot
Multi-step Agent	LLM controls iteration and continuation of the loop	Research agent that keeps digging
Multi-Agent	One agentic workflow can call another agent	Manager agent + worker agents

2. LLM Recap

- LLMs are mostly **Transformer decoders** trained to predict the *next token*.
- Special tokens** mark structure — e.g. `<|im_start|>`, `<|im_end|>`, `EOS` — different per model family.
- A **chat template** converts a list of role-tagged messages (system/user/assistant) into the exact string format the base model expects.

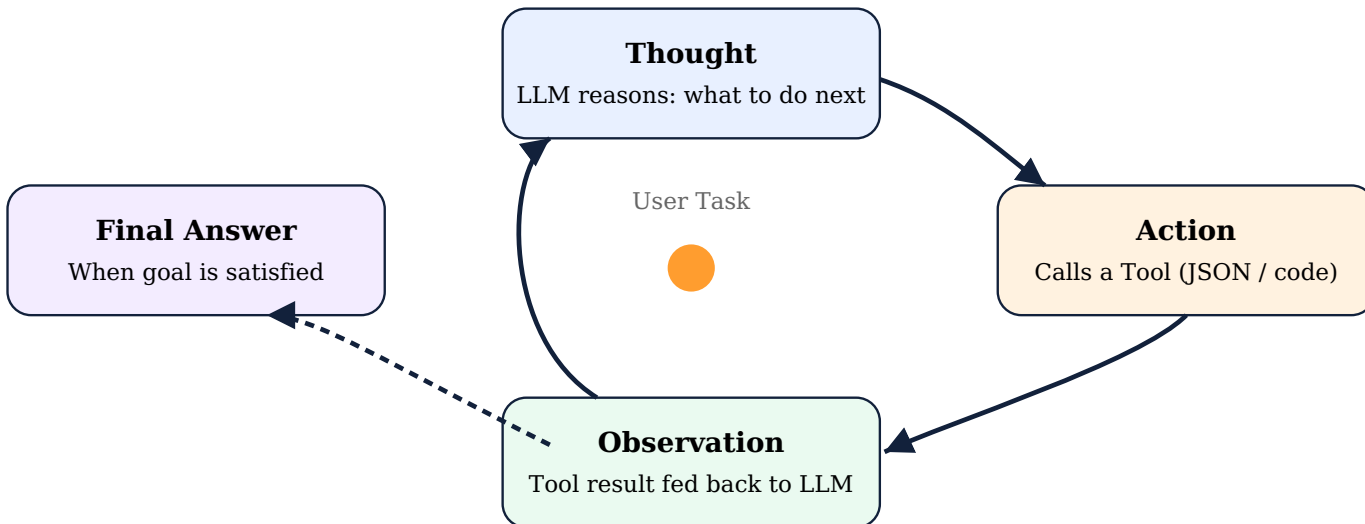
3. Messages & Chat Templates (example)

```
messages = [
    {"role": "system", "content": "You are a helpful travel assistant."},
    {"role": "user", "content": "Find me a hotel in Paris."}
]
# tokenizer.apply_chat_template(messages, tokenize=False)
# -> turns this into model-specific special-token formatted text
```

4. Tools

A Tool is a function given to the LLM with a **name**, **description**, **input schema (types)**, and **output**. The LLM never "calls" the tool directly — it *writes* text requesting a call; the agent framework parses that text and actually executes the function, then feeds the result back in.

5. The Agent Cycle: Thought → Action → Observation (ReAct)



Loop repeats: **Thought → Action → Observation → Thought → ...** until the LLM decides it has enough information to produce a **Final Answer**. This is the "ReAct" pattern (Reason + Act).

6. Action Formats

Format	How it works	Used by
JSON Agent	LLM outputs a JSON blob: <code>{"action": "get_weather", "action_input": {"city": "Paris"}}</code>	Most tool-calling frameworks
Code Agent	LLM writes actual executable code (e.g. Python) that calls tools directly	smolagents (CodeAgent)

7. System Prompt shapes everything

The system prompt defines: available tools (name/description/args), the expected output format, and the goal/persona. It's the single most important lever for agent behavior.

DEEP DIVE

Base model vs Instruct model

Type	Trained on	Behavior
Base model	Raw next-token prediction on huge text corpora	Completes text; doesn't "follow instructions" naturally
Instruct model	Base model + fine-tuning on (instruction, response) pairs	Follows chat-style prompts; understands system/user/assistant roles

Agents are almost always built on **instruct models** because the whole loop depends on the model reliably following a structured format turn after turn. **Chat template formats differ per model family** — this is why the same messages list produces different raw text depending on the model:

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
What's the weather?<|im_end|>
<|im_start|>assistant
    (ChatML format — used by many OpenAI-style models)

<s>[INST] You are a helpful assistant.
What's the weather? [/INST]
    (Llama/Mistral-style INST format)
```

This is exactly why `tokenizer.apply_chat_template()` exists — you write one messages list, and it auto-converts to whichever raw format the loaded model expects. Using the wrong template silently degrades agent performance because the model sees "unfamiliar" text.

How "the LLM calls a tool" really works, step by step

- Tool definitions (name, description, JSON-schema of args) are serialized into the system prompt.
- The LLM generates plain text that matches the requested action format (a JSON blob, or a code block).
- The agent framework **parses** that text with a regex/JSON-parser/code-executor — this is *not* magic, it's string parsing you could write yourself (as Unit 1's "dummy agent" shows).
- The framework executes the real Python function matching the requested tool name + args.
- The return value is serialized back into a new "tool"/"observation" message and appended to the conversation.
- The full updated message list is sent back to the LLM for the next Thought.

Stopping conditions for the loop

- The LLM emits a designated "Final Answer" / `final_answer()` action.
- A **max_steps / max_iterations** limit is hit (safety net against infinite loops).
- A timeout or token-budget ceiling is reached.

JSON Agents vs Code Agents — trade-offs

	JSON Agent	Code Agent
Parsing	Simple, very robust (schema-validated)	Needs a code sandbox/interpreter
Multi-tool chaining	1 LLM call per tool call	Many tool calls in a single code block → fewer LLM calls
Expressiveness	Limited to what the schema allows	Full language power — loops, conditionals, variables
Safety	Easy to sandbox (no arbitrary code)	Needs real sandboxing (E2B/Docker) in production

EXAMPLE

A minimal manual "dummy agent" (no framework) — the model outputs a Thought + Action, the Python code parses it, executes the tool, and appends the Observation back into the prompt:

```
def get_weather(city: str) -> str:
    return f"The weather in {city} is sunny, 22°C."

# LLM output (as text):
# Thought: I need the weather for Paris.
# Action: get_weather(city="Paris")
```

```
# Agent code parses the action, calls the real function:
observation = get_weather("Paris")
# Feed observation back as next message, LLM continues reasoning
# Thought: I now know the weather.
# Final Answer: It's sunny and 22°C in Paris.
```

DEVELOPER NOTES

- Always give tools a **precise docstring** — the LLM only "knows" a tool through its description.
- Keep the tool output **short and structured**; huge raw responses waste context and confuse reasoning.
- Code Agents (smolagents-style) tend to need fewer LLM calls than JSON agents for multi-step tasks, since one code block can chain several tool calls at once.

KEY TAKEAWAY

Every agent, in every framework, is running the same loop: **reason → act via a tool → observe → repeat → answer**. Master this mental model once, and every framework becomes "just an implementation detail."

COMMON MISTAKES

- Writing vague tool descriptions → LLM picks the wrong tool or wrong arguments.
- Forgetting to cap the number of loop iterations → infinite Thought/Action loops on hard tasks.
- Confusing "the LLM calls the tool" with reality — the LLM only *requests* a call in text; your code executes it.

QUICK REVISION

Agent = LLM + Tools + Loop · Cycle = Thought → Action → Observation → repeat → Final Answer · Actions = JSON blob or executable Code · System prompt defines available tools + format.

Frameworks for AI Agents — Overview

How the Thought→Action→Observation loop is implemented across 3 popular libraries.

OVERVIEW

You don't need a framework to build an agent (Unit 1 proved that), but frameworks remove boilerplate: LLM call plumbing, tool-calling parsing, memory, error handling, and multi-agent orchestration. This unit tours three: **smolagents** (lightweight, code-first, by Hugging Face), **LlamaIndex** (data + retrieval-centric, workflow-based), and **LangGraph** (explicit control-flow via graphs, by LangChain).

CORE CONCEPTS

Framework	Philosophy	Best for	
smolagents	Minimal code, agent writes/executes Python to act	Fast prototyping, code-centric tasks	
LlamaIndex	Agents built around indexes/retrieval + event-driven Workflows	RAG, agents over your own data	
LangGraph	Explicit state machine / graph of nodes & edges	Production apps needing fine control, branching, human-in-the-loop	

Dimension	smolagents	LlamaIndex	LangGraph
Learning curve	Low	Medium	Medium-High
Control over flow	Low (LLM decides most of it)	Medium (Workflows add structure)	High (you define the graph)
Memory / state	Simple step memory	Chat memory + context objects	Explicit typed State + checkpointer
Multi-agent support	Manager agent calling sub-agents	Multi-agent Workflows	Supervisor / swarm graph patterns
Ecosystem strength	Hugging Face Hub, Spaces	LlamaHub connectors, RAG tooling	LangChain ecosystem, LangSmith tracing

KEY TAKEAWAY

Pick the framework based on **how much control you need over the flow**: smolagents = fastest to write, LangGraph = most controllable, LlamaIndex = best when data/retrieval is central. In real projects, teams often combine them — e.g. LlamaIndex for retrieval quality inside a LangGraph node for overall control.

smolagents — Code-first Agents

A lightweight Hugging Face library where the agent reasons in executable Python.

OVERVIEW

smolagents is a minimal agent library where, instead of outputting a JSON action, the LLM writes a small **Python code snippet** that is executed (in a sandboxed interpreter) to call tools, do calculations, or chain steps — reducing the number of LLM calls needed per task.

CORE CONCEPTS

Main agent classes

Class	Action format	Notes
<code>CodeAgent</code>	Writes & executes Python code	Default, most flexible, fewer steps
<code>ToolCallingAgent</code>	Writes JSON tool calls	Closer to "standard" tool-calling agents

Defining a Tool — use the `@tool` decorator (function must have type hints + a docstring — smolagents parses these into the tool schema shown to the LLM):

```
from smolagents import tool, CodeAgent, InferenceClientModel

@tool
def get_weather(city: str) -> str:
    """Fetches the current weather for a given city.

    Args:
        city: Name of the city.
    """
    return f"The weather in {city} is sunny, 22°C."

agent = CodeAgent(tools=[get_weather], model=InferenceClientModel())
agent.run("What's the weather in Tokyo, and should I bring an umbrella?")
```

Other building blocks

- **Models:** `InferenceClientModel` (HF Inference), or wrap OpenAI/local models.
- **Multi-agent systems:** a "manager" `CodeAgent` can call other agents as tools.
- **Vision agents:** pass images into the agent's context for visual tasks (e.g. browser screenshots).
- **Retrieval agents:** combine a retriever tool with `CodeAgent` for agentic RAG.
- **Sandboxing:** code execution can run locally (restricted builtins) or in E2B/Docker sandboxes for safety.

DEEP DIVE

Two ways to define a Tool

Method	When to use
<code>@tool</code> decorator on a function	Quick, stateless tools — most common case
Subclass <code>Tool</code> (with <code>name</code> , <code>description</code> , <code>inputs</code> , <code>output_type</code> , <code>forward()</code>)	Stateful tools, or tools needing setup (e.g. loading a model once)

```
from smolagents import Tool

class WeatherTool(Tool):
    name = "get_weather"
    description = "Fetches current weather for a city."
    inputs = {"city": {"type": "string", "description": "City name"}}
    output_type = "string"

    def forward(self, city: str) -> str:
        return f"The weather in {city} is sunny, 22°C."
```

Agent execution loop internals

- Each step: the agent builds a prompt from the **system prompt + tool specs + memory of past steps**, calls the model, parses the generated code/JSON, executes it, and stores the result as a new memory step.
- `agent.run()` loops this until `final_answer()` is called or `max_steps` is reached.
- **Planning steps:** smolagents can periodically insert a "planning" step where the model re-summarizes progress and re-plans — useful for long, multi-stage tasks.

Multi-agent pattern (manager + workers)

```
web_agent = CodeAgent(tools=[web_search, visit_page], model=model, name="web_agent",
                      description="Browses the web to answer questions.")

manager_agent = CodeAgent(tools=[], managed_agents=[web_agent], model=model)
```

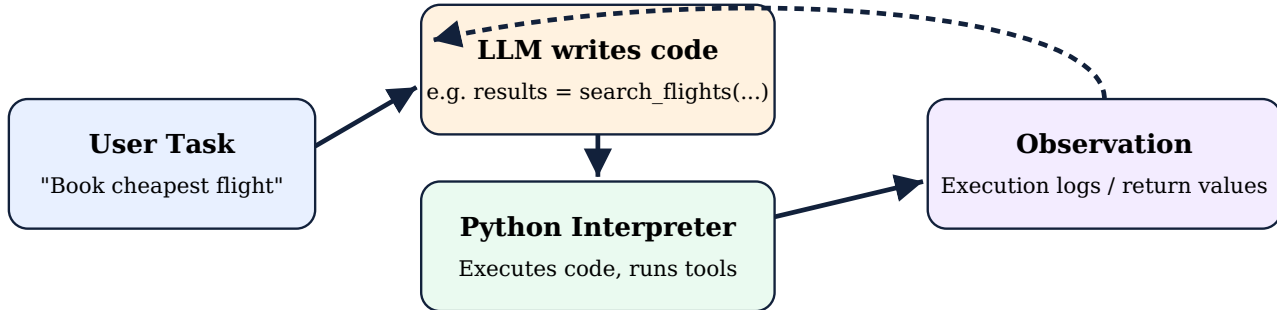


```
manager_agent.run("Find the most recent Nobel Prize winners and summarize their work.")
# manager_agent treats web_agent as a callable "tool" it can delegate sub-tasks to
```

Vision & multimodal agents — pass PIL images directly into `agent.run(task, images=[img])` ; useful for browser-automation agents that "look" at screenshots to decide the next click/action. **Sandboxing options, from least to most isolated**

1. **Local Python executor** with restricted builtins (fast, learning/demo only).
2. **E2B sandbox** — remote, ephemeral micro-VM, good balance of speed + isolation.
3. **Docker sandbox** — full container isolation, best for untrusted/production workloads.

Sharing agents: `agent.push_to_hub("username/my-agent")` uploads the agent (tools + config) as a Hugging Face Space others can load with `agent.from_hub(...)`.



EXAMPLE

Multi-tool CodeAgent that reasons across two tools in a single generated code block (this is the key efficiency win vs. JSON agents, which need one LLM call per tool call):

```
results = web_search(query="best pizza in Naples")
top = results[0]
directions = get_directions(destination=top["address"])
final_answer(f"Try {top['name']}. Directions: {directions}")
```

DEVELOPER NOTES

- Type hints + docstrings are **not optional** — smolagents literally builds the tool spec from them.
- Use a sandbox (E2B/Docker) in production; raw local code execution from an LLM is a security risk.
- Push/pull agents to the Hugging Face Hub for sharing (Spaces integration).

KEY TAKEAWAY

smolagents trades "safety guardrails" for **speed and flexibility** — code as the action language lets one LLM turn accomplish what would take several JSON-agent turns.

COMMON MISTAKES

- Missing/incorrect type hints on `@tool` functions → schema errors at runtime.
- Running agent-generated code without sandboxing outside of learning/demo contexts.
- Overloading one agent with too many unrelated tools instead of splitting into a multi-agent system.

QUICK REVISION

smolagents = HF's lightweight lib · CodeAgent (writes+runs Python) vs ToolCallingAgent (writes JSON) · `@tool` decorator + docstring = schema · supports multi-agent, vision, retrieval, sandboxing.

LlamaIndex — Agents Over Your Data

Build LLM-powered agents around indexes, retrieval, and event-driven Workflows.

OVERVIEW

LlamaIndex is designed around connecting LLMs to **your own data** — documents, PDFs, APIs, SQL — via **Components → Tools → Agents → Workflows**. It shines for RAG-heavy and enterprise-data agents.

CORE CONCEPTS

Layer	Purpose
Components	Building blocks — LLMs, prompts, retrievers, readers
QueryEngine / Index	Turns raw documents into a searchable vector/graph index
Tools	Wraps a QueryEngine or function so an Agent can call it
Agents	<code>FunctionAgent</code> / <code>ReActAgent</code> reason over tools
Workflows	Event-driven orchestration — chain steps, branch, run agents in sequence

Typical pipeline: Load documents (readers) → build a **VectorStoreIndex** → wrap it as a **QueryEngineTool** → give it to an **Agent** → wire multiple agents/steps together in a **Workflow** for complex multi-stage tasks.

DEEP DIVE

Components breakdown

Component	Role
LLM	The reasoning/generation model (OpenAI, HF Inference, local, etc)
Embedding model	Converts text chunks into vectors for similarity search
Node parser / Splitter	Breaks documents into "Nodes" (chunks) with metadata
Storage context	Where vectors/docs live — in-memory, Chroma, Pinecone, Qdrant, etc.
Retriever	Fetches top-k relevant Nodes for a query from the index

Query Engine types

Type	Best for
VectorStoreIndex query engine	Semantic similarity search over chunks (most common)
SummaryIndex query engine	Summarizing across an entire document/corpus
RouterQueryEngine	Picks between multiple query engines depending on the question
SubQuestionQueryEngine	Splits a complex question into sub-questions, answers each, combines

Agent types in LlamaIndex

- `FunctionAgent` — modern, function/tool-calling style agent (recommended default).
- `ReActAgent` — explicit Thought/Action/Observation text-based reasoning, works with any LLM (even ones without native tool-calling).
- `AgentWorkflow` — orchestrates **multiple agents** together (e.g. a router agent handing off to specialist agents).

Workflows — the event-driven engine

```
from llama_index.core.workflow import Workflow, step, StartEvent, StopEvent, Event

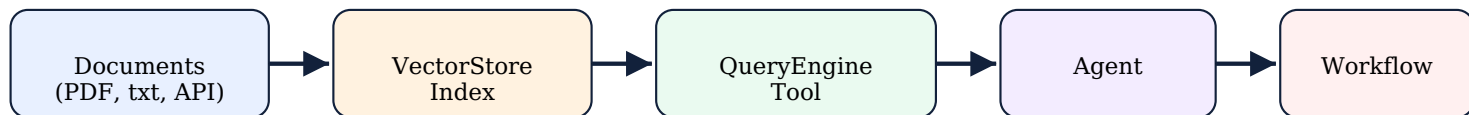
class RetrieveEvent(Event):
    query: str

class MyWorkflow(Workflow):
    @step
    async def start(self, ev: StartEvent) -> RetrieveEvent:
        return RetrieveEvent(query=ev.query)

    @step
    async def retrieve_and_answer(self, ev: RetrieveEvent) -> StopEvent:
        result = query_tool.call(ev.query)
        return StopEvent(result=result)

workflow = MyWorkflow(timeout=60)
result = await workflow.run(query="What is our refund policy?")
```

- Each `@step` function declares which **Event** types it consumes and emits — LlamaIndex wires the graph automatically from these type hints.
- Supports **branching, loops, parallel steps**, and streaming intermediate events back to a UI.
- Great for multi-stage pipelines: e.g. retrieve → verify → re-retrieve if confidence is low → answer.



EXAMPLE

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
from llama_index.core.tools import QueryEngineTool
from llama_index.core.agent.workflow import FunctionAgent

docs = SimpleDirectoryReader("data/").load_data()
index = VectorStoreIndex.from_documents(docs)
query_tool = QueryEngineTool.from_defaults(
    query_engine=index.as_query_engine(),
    name="company_docs",
    description="Answers questions about our internal company documents."
)

agent = FunctionAgent(tools=[query_tool], llm=llm)
response = await agent.run("What is our refund policy?")
```

DEVELOPER NOTES

- Chunk size & overlap when indexing directly affect retrieval quality — tune per document type.
- Use **Workflows** (event-driven, step decorators) once you need branching logic beyond a single agent loop.
- LlamaHub has hundreds of pre-built readers/tools (Notion, Slack, SQL, Google Drive, etc).

KEY TAKEAWAY

LlamaIndex = "agent whose superpower is retrieval." Use it when the hard part of your task is **finding the right information**, not just calling arbitrary tools.

COMMON MISTAKES

- Skipping chunking strategy tuning → poor retrieval, hallucinated answers.
- Giving the agent a vague tool description for the QueryEngineTool → it doesn't know when to use it.
- Not evaluating retrieval quality separately from generation quality.

QUICK REVISION

Components → Index → QueryEngineTool → Agent → Workflow. Best for RAG / agents-over-data.

LangGraph — Controllable Agent Graphs

Model your agent as an explicit graph of nodes and edges for production-grade control.

OVERVIEW

LangGraph represents an agent as a **graph**: **State** (shared data), **Nodes** (functions/LLM calls that update state), and **Edges** (control flow, including conditional branching). Unlike a free "let the LLM decide everything" loop, you can enforce exact structure — critical for production reliability.

CORE CONCEPTS

Concept	Role
State	A typed dict/object shared and updated across the whole graph run
Nodes	Python functions — call an LLM, run a tool, transform state
Edges	Fixed transitions between nodes
Conditional Edges	Function decides the next node based on current state (branching)
START / END	Special entry and exit points of the graph
Checkpoint	Persists state → enables memory, pausing, human-in-the-loop

DEEP DIVE

State reducers — control *how* a node's return value merges into shared state instead of overwriting it:

```
from typing import Annotated
from langgraph.graph.message import add_messages

class State(TypedDict):
    messages: Annotated[list, add_messages] # appends instead of replacing
    step_count: int
```

Without a reducer, returning `{"messages": [...]}` from a node **replaces** the whole list; `add_messages` instead appends new messages — critical for keeping conversation history intact. **Prebuilt components**

- `ToolNode` — a ready-made node that executes any pending tool calls from the last AI message.
- `tools_condition` — a ready-made conditional-edge function ("does the last message contain a tool call?").
- `create_react_agent()` — one-liner that builds a full ReAct-style graph (model + ToolNode + loop) for simple cases.

Human-in-the-loop with `interrupt()`

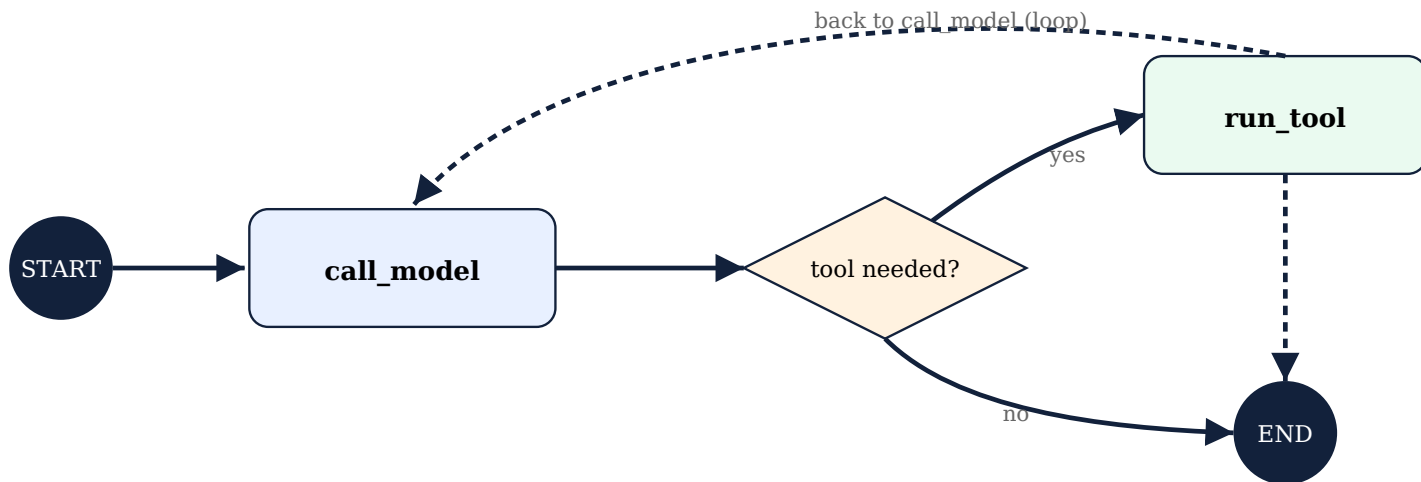
```
from langgraph.types import interrupt

def confirm_send_email(state: State):
    approval = interrupt({"question": "Send this email?", "draft": state["draft"]})
    if approval != "yes":
        return {"status": "cancelled"}
    return {"status": "sent"}
```

Execution pauses at the interrupt, state is checkpointed, and a human (or another system) resumes the graph later with an answer — without losing any progress. **Multi-agent patterns in LangGraph**

Pattern	Description
Supervisor	One "router" node decides which specialist agent-node handles the next step
Swarm	Agents hand off control to each other directly, no central router
Subgraphs	A whole graph is nested inside a node of a bigger graph — enables modular reuse

Streaming: `app.stream(...)` yields intermediate state updates as the graph runs — used to show live "agent is thinking / calling tool X" UI feedback.



EXAMPLE

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

class State(TypedDict):
    messages: list

def call_model(state: State):
    response = llm.invoke(state["messages"])
    return {"messages": state["messages"] + [response]}

def should_continue(state: State):
    last = state["messages"][-1]
    return "tools" if last.tool_calls else END

graph = StateGraph(State)
graph.add_node("agent", call_model)
graph.add_node("tools", run_tools)
graph.add_edge(START, "agent")
graph.add_conditional_edges("agent", should_continue, {"tools": "tools", END: END})
graph.add_edge("tools", "agent")
app = graph.compile()

app.invoke({"messages": [user_message]})
```

DEVELOPER NOTES

- Use a **checkpoint** (e.g. `MemorySaver`) to persist state across turns — enables multi-turn conversations and resuming after a crash.
- LangGraph Studio gives a visual debugger for the graph — invaluable for complex flows.
- Add a "human-in-the-loop" node before risky actions (e.g. sending an email) by pausing at a checkpoint.

KEY TAKEAWAY

LangGraph gives up some of the LLM's "free will" over control flow in exchange for **predictability, debuggability, and safety** — the right trade-off for production agents.

COMMON MISTAKES

- Forgetting the loop-back edge (tools → agent) → agent never re-reasons after a tool call.
- Mutating state incorrectly (not returning proper partial updates) → silent bugs.
- Overusing conditional edges instead of simplifying state → hard-to-debug spaghetti graphs.

QUICK REVISION

State + Nodes + Edges (+ Conditional Edges) = Graph. START/END mark boundaries. Checkpointer = memory / pause / resume. Best for controllable, production agents.

Use Case — Agentic RAG

Combining retrieval with an agent loop so the agent decides when/what/how to retrieve.

OVERVIEW

Classic RAG = retrieve once → stuff into prompt → generate. **Agentic RAG** instead treats retrieval as a **Tool** the agent can call zero, one, or many times, and can combine with other tools (web search, calculators, calendar APIs) — e.g. building a "party/event planning" assistant across Unit 3.

CORE CONCEPTS

- **Retriever-as-tool:** wrap a vector index / search API behind a tool interface with a clear description.
- **Multi-source retrieval:** agent can pick between a knowledge-base tool, a web-search tool, or a guest-list/API tool based on the question.
- **Query rewriting:** the agent can reformulate the user's question before retrieving, improving recall.
- **Iterative retrieval:** if the first retrieval isn't enough, the agent can retrieve again with a refined query (unlike static RAG).

DEEP DIVE

Course walkthrough — "Alfred", the gala assistant

Unit 3 builds up a single running project step by step, adding one capability at a time:

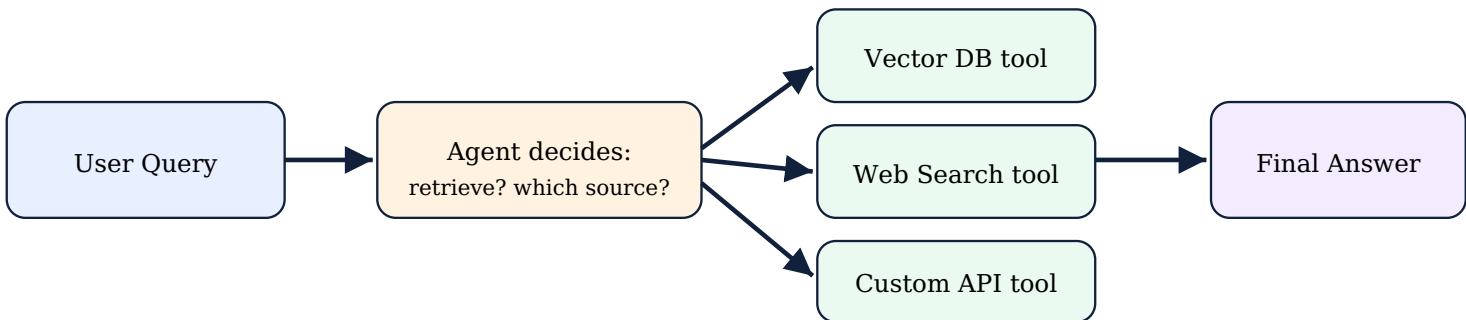
1. **Guest info retrieval tool** — load a small structured dataset (name, relation, notes) and expose a `guest_info_retriever` tool that does keyword/semantic search over it.
2. **Web search tool** — for questions the guest dataset can't answer (e.g. "what's this guest famous for").
3. **Weather tool** — a custom function-based tool for a completely different data source, showing how tools can be freely mixed regardless of where their data comes from.
4. **Hub stats tool** — pulls live stats from the Hugging Face Hub API, showing real external-API integration.
5. **Structured outputs** — constrain the final answer to a fixed schema (e.g. a toast-speech format) so downstream code can reliably parse the agent's output.

Why build retrieval as a Tool instead of always-on RAG?

- Cheaper: no retrieval call wasted on questions that don't need it (e.g. "what's 12 × 8?").
- More accurate: the agent can choose a *different* source per question instead of one fixed index.
- Composable: retrieval becomes just one tool among many, so it naturally combines with calculators, APIs, etc.

Embedding & indexing choices that matter

Choice	Impact
Chunk size	Too small → loses context; too large → noisy, less precise matches
Embedding model	Domain-specific models beat generic ones for jargon-heavy data
Top-k	Too low → misses info; too high → floods the LLM's context with noise
Metadata filters	Pre-filter by date/category before vector search for precision



EXAMPLE

Course's running example: a **Gala/party-planning assistant** that retrieves guest info from a knowledge base tool, checks the web for current news, and uses a weather tool — all through one agent:

```
@tool
def guest_info_retriever(query: str) -> str:
    """Retrieve details about a gala guest by name or role."""
    return guest_index.query(query)

agent = CodeAgent(tools=[guest_info_retriever, web_search, weather_tool], model=model)
agent.run("Should I mention Ada Lovelace's achievements in my toast, and will it rain during the gala?")
```

DEVELOPER NOTES

- Give each retrieval tool a **distinct, non-overlapping description** so the agent picks correctly.
- Log which tool was called for which query — essential for debugging agentic RAG.

KEY TAKEAWAY

Agentic RAG = RAG where **retrieval itself becomes a decision**, not a fixed pipeline step — more flexible, but needs good tool descriptions to stay accurate.

QUICK REVISION

Retrieval as a tool → agent decides when/what/how many times to retrieve → can mix multiple sources.

Final Project — GAIA Benchmark Agent

Build and submit your own agent, evaluated against the GAIA benchmark, for certification.

OVERVIEW

The capstone: build your own agent (any framework you like) and evaluate it on a subset of **GAIA** (General AI Assistants benchmark) — real-world questions requiring reasoning, tool use, web browsing, and multi-modal understanding. Score $\geq 30\%$ to earn the Certificate of Completion.

CORE CONCEPTS

- **GAIA** tests agents on tasks humans find easy but that require real tool use — web search, file parsing, spreadsheet reading, multi-hop reasoning — not just knowledge recall.
- You submit your agent's answers, get scored, and can appear on the **Student Leaderboard**.
- Any framework is allowed — smolagents/LangGraph/LlamaIndex/custom — the goal is a working, robust agent.

DEEP DIVE

What makes GAIA hard

- Questions require **multi-hop reasoning** — the answer isn't in one document, it needs combining several retrieved/tool-derived facts.
- Many questions need **real tool execution**: opening a file, reading a spreadsheet cell, running a calculation, reading an image — not something the LLM can "just know."
- Answers must be given in an **exact expected format** (e.g. a number, a short string) — verbose or slightly-off answers are scored wrong even if the reasoning was correct.

Recommended build order

1. Start from the official final-agent template Space (clone it into your own Hub account).
2. Get a minimal agent running end-to-end on 2-3 easy questions first — verify the submission pipeline works.
3. Add tools incrementally: web search → page visitor → file reader (pdf/xlsx/csv) → python execution → image reasoning.
4. Add an explicit "final answer formatter" step so raw agent output gets cleaned into GAIA's expected format.
5. Run the full evaluation subset, inspect failures one by one, and patch the specific tool/prompt that failed.

Submission & scoring

- Your agent answers a fixed subset of GAIA questions; answers are auto-scored for exact/near-exact match.
- $\geq 30\%$ correct → Certificate of Completion (combined with Unit 1 + a use-case assignment).
- Optional: submit your score to the public **Student Leaderboard** to compare with other learners.

EXAMPLE

Typical final-agent tool stack: `web_search` `page_visit` `python_interpreter` `file_reader (pdf/xlsx/csv)` `image_analysis` `calculator` — combined in a CodeAgent or LangGraph loop with generous iteration limits for multi-hop questions.

DEVELOPER NOTES

- Add strong **error handling/retries** around tools — GAIA questions often fail silently on flaky web tools.
- Log every Thought/Action/Observation for the run — makes debugging failed questions far easier.
- Start from the official "final agent" template Space before customizing.

KEY TAKEAWAY

This unit forces you to combine *everything* — reasoning loop, multiple tools, retrieval, error handling — into one robust system. It's the real test of whether the fundamentals actually stuck.

QUICK REVISION

Build agent → run on GAIA subset → need $\geq 30\%$ to certify → any framework allowed → submit to leaderboard.

Fine-tuning an LLM for Function-calling

Teach a smaller model to reliably emit tool calls instead of relying on prompting alone.

OVERVIEW

Not every model is naturally good at outputting well-formed tool calls. This bonus unit fine-tunes a base LLM (with **LoRA**, parameter-efficient fine-tuning) on a dataset of function-calling examples, so it reliably produces the special tokens/format an agent framework expects.

CORE CONCEPTS

- **Dataset format:** conversations with a Thought + a structured function call + Observation, e.g. using special tokens like `<think>` and `<tool_call>`.
- **LoRA** (Low-Rank Adaptation): freeze base weights, train small rank-decomposition adapter matrices — cheap to fine-tune, easy to swap in/out.
- **Chat template updates:** new special tokens must be added to the tokenizer + chat template so the model consistently produces the right format at inference.
- Trained typically via `trl`'s `SFTTrainer` on Hugging Face.

Example training sample (simplified)

```
{
  "messages": [
    {"role": "user", "content": "What's the weather in Paris?"},
    {"role": "assistant", "content":
      "I need current weather, I'll call the weather tool.
      {\\"name\\": \\"get_weather\\", \\"arguments\\": {\\"city\\": \\"Paris\\"}}"},
    {"role": "tool", "content": "Sunny, 22°C"},
    {"role": "assistant", "content": "It's sunny and 22°C in Paris right now."}
  ]
}
```

LoRA config essentials

Param	Meaning
<code>r</code> (rank)	Size of the adapter matrices — higher = more capacity, more memory
<code>lora_alpha</code>	Scaling factor applied to the adapter's contribution
<code>target_modules</code>	Which layers get adapters (commonly attention projection layers)
<code>lora_dropout</code>	Regularization to prevent overfitting on small datasets

Minimal training sketch

```
from peft import LoraConfig
from trl import SFTTrainer, SFTConfig

lora_config = LoraConfig(r=16, lora_alpha=32, target_modules=["q_proj", "v_proj"])
trainer = SFTTrainer(
    model=base_model,
    train_dataset=function_calling_dataset,
    peft_config=lora_config,
    args=SFTConfig(output_dir="ft-function-calling", num_train_epochs=3),
)
trainer.train()
trainer.push_to_hub()
```

After training: merge the LoRA adapter into the base model (or keep it separate and load with `PeftModel.from_pretrained`), then plug the fine-tuned model straight back into any framework from Unit 2 as the agent's "brain."

DEVELOPER NOTES

Fine-tuning is worth it when: prompting alone gives inconsistent tool-call formatting, you need a smaller/cheaper model to match a bigger model's tool-use reliability, or you have a narrow, repeatable tool-use domain (e.g. one company's internal APIs). Always keep a held-out eval set of function-calling examples to check the fine-tune actually improved formatting accuracy, not just memorized training data.

KEY TAKEAWAY

Prompting gets you far; fine-tuning gets you **consistency** — useful once you need a smaller/cheaper model to behave as reliably as a frontier model on tool-calling.

QUICK REVISION

Function-calling dataset + LoRA fine-tuning + updated chat template/special tokens = reliable tool-callers.

Agent Observability & Evaluation

Trace what your agent actually did, and measure whether it's getting better or worse.

OVERVIEW

Once an agent is running loops and calling tools, "print debugging" doesn't scale. This unit covers **tracing** (recording every Thought/Action/Observation/LLM-call as spans) and **evaluation** (scoring agent runs against test sets), typically via **OpenTelemetry** instrumentation and tools like **Langfuse**.

CORE CONCEPTS

- **Tracing:** every LLM call, tool call, and intermediate step is logged as a span with inputs/outputs/latency/cost.
- **OpenTelemetry (OTel):** vendor-neutral standard for exporting these traces to an observability backend.
- **Evaluation datasets:** a fixed set of tasks + expected outcomes used to regression-test agent changes.
- **Metrics:** task success rate, number of steps taken, tool-call accuracy, latency, cost per run.

Instrumenting with Langfuse (typical setup)

```
from langfuse import Langfuse
from openinference.instrumentation.smolagents import SmolagentsInstrumentor

SmolagentsInstrumentor().instrument() # auto-captures every agent step as an OTel span

agent.run("Plan a 3-day trip to Kyoto")
# Every Thought/Action/Observation now shows up as a trace in the Langfuse dashboard,
# with latency, token usage, and cost per step.
```

Online vs offline evaluation

Type	When	Example
Offline eval	Before deploying / after every change	Run a fixed 50-question test set, compare pass rate to baseline
Online eval	Continuously in production	Sample live traces, score with an LLM-as-judge or user feedback (👍/👎)

LLM-as-judge pattern — use a second LLM call to score the agent's final answer against a rubric or reference answer, producing a numeric/pass-fail score you can track over time and across model/prompt versions. **What good traces let you debug**

- Which exact tool call failed, and with what arguments and error message.
- Whether the agent looped unnecessarily (repeated near-identical tool calls).
- Cost/latency hotspots — e.g. one slow tool dominating total run time.

DEVELOPER NOTES

Instrument early — retrofitting tracing onto a "working" agent is much harder once it's in production. Treat your eval set like unit tests: run it after every prompt/tool/model change, and version it alongside your agent code so regressions are traceable to a specific commit.

KEY TAKEAWAY

"It worked in my one test" ≠ reliable. Observability + a real eval set turn agent development from vibes-based debugging into an actual engineering discipline.

QUICK REVISION

Trace every step (OTel/Langfuse) + maintain an eval dataset + track success rate/steps/cost over time.

Agents in Games — Pokémon Battles

A fun, complex real-time environment to stress-test agent reasoning and action-taking.

OVERVIEW

This unit builds an agent that plays **Pokémon battles** — a great testbed because it needs state-tracking (HP, types, status effects), strategic multi-step planning, and fast decision-making under a constrained action space (a handful of valid moves per turn).

CORE CONCEPTS

- **Game state as agent context:** current HP, type matchups, move list must be fed into the prompt each turn.
- **Constrained action space:** unlike open-ended tool use, the agent must pick from a small valid move set — good practice for "structured output" agents.
- **Type-effectiveness reasoning:** tests whether the LLM can apply domain rules (e.g. water beats fire) consistently across many turns.

Architecture sketch

1. A game-state extractor reads the current battle state (HP%, active Pokémon, move list, status effects) from the emulator/API.
2. That state is serialized into a compact text/JSON block and injected into the agent's prompt each turn.
3. The agent reasons over type matchups and move power, then picks **one** action from the valid move list — this is a textbook case of **constrained generation** (the output space is tiny and enumerable).
4. The chosen move is sent back to the game; the resulting new state becomes next turn's Observation.

Why this stresses agents differently than GAIA-style tasks

- Very **high turn count** per episode — small reasoning errors compound over a full battle.
- State changes **every single turn**, unlike most tool-use tasks where context is mostly static.
- Tight, real-time-ish loop — a good testbed for measuring an agent's per-step latency and reliability under repetition.

KEY TAKEAWAY

Games are a low-stakes, highly measurable way to test whether your agent's reasoning loop actually holds up over many sequential decisions — skills that transfer directly to real multi-step agent tasks.

QUICK REVISION

Structured state in → constrained valid moves out → win/loss = an easy, objective agent benchmark.

Whole Course — One Page

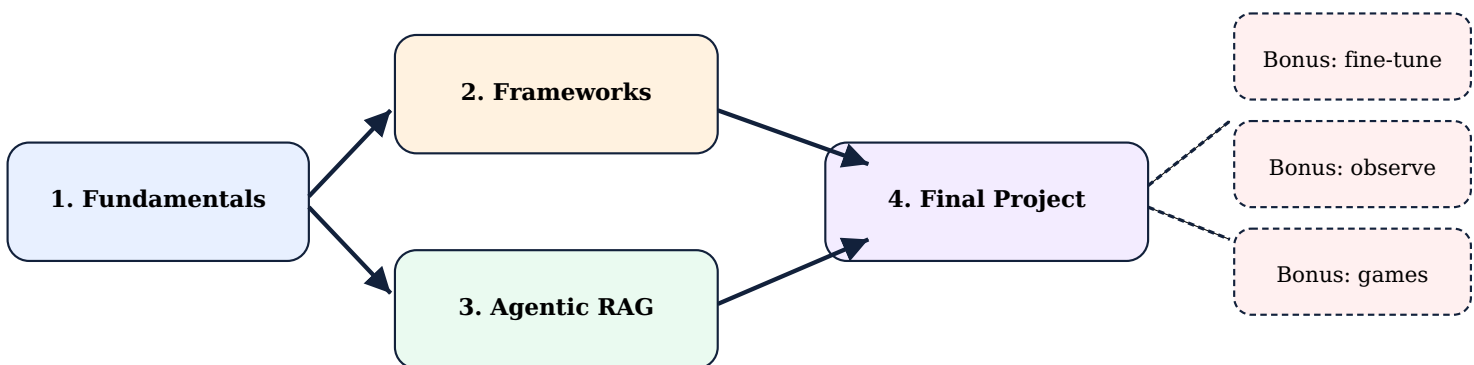
For last-minute revision before an interview, quiz, or project.

COURSE MAP

Unit	One-liner
0. Onboarding	Syllabus, certification paths, tools/account setup
1. Fundamentals	Agent = LLM + Tools + Thought→Action→Observation loop
2. Frameworks	smolagents (code-first) · LlamaIndex (data-first) · LangGraph (control-first)
3. Agentic RAG	Retrieval becomes a tool the agent chooses to call
4. Final Project	Build + evaluate an agent on GAIA, ≥30% to certify
B1. Fine-tuning	LoRA-tune a model for consistent function-calling output
B2. Observability	Trace every step (OTel/Langfuse) + maintain eval sets
B3. Games	Pokémon battles as a structured, measurable agent benchmark

CORE VOCABULARY

Term	Meaning
Agent	LLM + tools + a loop that lets it reason and act repeatedly
Tool	A function exposed to the LLM with name/description/typed args
Thought	The LLM's internal reasoning about what to do next
Action	The LLM's request to call a tool (JSON or code)
Observation	The result of executing the action, fed back to the LLM
ReAct	The "Reason + Act" prompting pattern behind the whole loop
Chat template	Converts role-tagged messages into a model's exact expected text format
System prompt	Defines persona, available tools, and expected output format
GAIA	Benchmark of real-world, tool-requiring questions used for the final project
LoRA	Efficient fine-tuning method that trains small adapter weights, not the full model
Tracing/Observability	Logging every step of an agent run for debugging & evaluation



FINAL REMINDER

Whatever framework you use later — **LangChain, CrewAI, AutoGen, OpenAI Agents SDK** — you're always looking for the same four things: **the LLM, the tools, the loop, and the state/memory**. Find those four in any new framework and you'll understand it in minutes.